BlackHoleEncounter / SolarSystem2

Script-by-Script Summary of the Codebase

A black-hole flyby N-body simulation project built around a REBOUND-based integration engine, with post-processing for orbital/hazard/temperature analysis and a separate Gaia DR3 astrometric-detectability analysis thread.

Generated 2026-07-08

Contents

1. Core Simulation Engine
- solar_system_bh_rebound25_patched.py

2. Master Configuration File
- input.yaml

3. Orbit & Physics Post-Processing
- post_orbit_compare_geocentric_fixed4.py
- estimate_BH_parameters_uncertainties.py
- estimate_BH_parameters_uncertainties_fast.py
- postprocess_belt_sizes_and_hazard.py
- rebuild_planets_run_deltas_from_npz.py
- summarize_earth_temperatures1.py
- time_dilation_tides.py

4. Archive & Data Utilities
- inspect_archive_hashes.py
- replay_archive.py
- convert_npz_to_csv.py
- count_planets_run_rows.py

5. Visualization & Animation
- animate_snapshots3D_1.py
- run_animation.py
- make_inner_3au_videos_from_archive.py
- plot_bh_perihelion.py
- gallery_heliocentric_viewer.py
- gallery_heliocentric_replot_viewer_progress_workbook.py

6. GAIA Astrometric Detectability Analysis
- GAIA_dr3_v1.py
- astrometric_shift.py
- astrometric_shift_option1.py
- centroid_motion.py
- detectable_shift_on_date.py
- distance_from_bh_on_first_detect.py
- overlay_bh_on_vr_image.py
- overlay_plot_v2.py

1. Core Simulation Engine

The single script that actually runs the N-body integration of the Sun, planets, Moon, black hole, and asteroid-belt tracers, and drives full parameter sweeps. Almost every other script in the project either consumes its output files or duplicates a fragment of its logic.

`solar_system_bh_rebound25_patched.py`

PURPOSE

The core simulation engine of the project. Builds and integrates a REBOUND N-body simulation of the Sun, 8 planets, the Moon, an incoming black hole on a hyperbolic trajectory, and an optional population of massless asteroid-belt tracer particles, then produces a large suite of derived data products (Excel workbooks, CSVs, plots, and animations) documenting how the flyby perturbs the solar system. Supports full parameter sweeps (grid search over BH periapsis distance, approach speed, inclination, node/argument-of-periapsis angles, and perihelion-timing offset) and can resume an interrupted sweep.

INPUTS

A YAML parameter file (positional CLI arg, e.g. input.yaml) containing: epoch (UTC ISO date), kernel (JPL/Skyfield ephemeris kernel basename, e.g. de440s), bh_mass_msun, bh_rp_au (periapsis distance, or a min,max,step range), bh_vinf_kms (hyperbolic excess velocity, or range), bh_tperi_offset_days (or range), bh_Omega_deg / bh_omega_deg / bh_inc_deg (orbital orientation angles, or ranges), dt_days, duration_days, dump_interval_days, output_interval_days, n_belt (asteroid tracer count), archive_enable / archive_interval_days / archive_filename, compare_baseline_bh_mass_msun (for a parallel BH-free comparison run), write_npz, write_mp4, write_uncertainty_csv, uncertainty_bodies / uncertainty_output_dir, size_slope / size_min_km / size_max_km / size_seed (belt size distribution), real_ge1km_count, per_object_impact_prob_yr. CLI flag --resume-dir continues a previous sweep. A Skyfield .bsp ephemeris supplies initial heliocentric state vectors for the Sun and planets at epoch.

OUTPUTS

Per parameter-combination subfolder under simulations/<timestamp>/<timestamp>__rp..__vinf..__inc..__toff..__Om..__om../, containing: a copy of the input YAML, an optional REBOUND SimulationArchive (__archive.bin), snapshot_<t>.npz per-body state files, belt tracer before/after CSVs, a planets_run_deltas.csv (8 planets + Moon, elements before/after), an __orbits__<rp>__<mass>.xlsx workbook (per-body position, velocity, heliocentric distance, BH tidal acceleration, and displacement vs. a no-BH baseline), an __bh_radec__<rp>__<mass>.xlsx workbook (BH geocentric RA/Dec and line-of-sight velocity vs. time), per-body uncertainty-input CSVs, heliocentric plots, belt scatter/histogram plots, a hazard summary text file, and (if ffmpeg is available) inner-solar-system MP4 animations.

KEY TECHNIQUES

REBOUND N-body integration with the IAS15 adaptive integrator. Initial planetary state vectors come from a Skyfield JPL ephemeris (de440s) at the given UTC epoch. The Moon is added as a simple fixed radial offset from Earth plus a tangential velocity kick (not a real lunar ephemeris). The black hole's initial state is derived analytically from classical hyperbolic orbital elements (periapsis, v-infinity): solves the hyperbolic Kepler equation via Newton-Raphson to get the true anomaly, then rotates the perifocal-frame state into the simulation frame with a standard Omega/i/omega Euler-angle rotation, selecting the inbound branch. Belt tracers get random Keplerian elements converted to Cartesian state via a planar Kepler formula. Orbital elements (a, e, inclination, perihelion q) are recovered from Cartesian state via vectorized specific-energy / eccentricity-vector formulas. 'Tidal acceleration' uses a simple point-mass approximation (2*G*M_BH/R^3 * r_helio), not a full tidal tensor; no explicit post-Newtonian/GR terms are added to the equations of motion -- perturbation is measured only as the difference against a parallel BH-free baseline integration. BH sky-plane RA/Dec is computed via a standard unit-vector-to-spherical-angle conversion for detectability/GAIA comparisons. Belt-impact hazard estimation uses a power-law size distribution plus a Poisson impact-rate model.

NOTES

This is the master/driver script of the whole pipeline. Run as 'python solar_system_bh_rebound25_patched.py input.yaml [--resume-dir simulations/<stamp>]'. Performs a full Cartesian product over all six BH parameter ranges, so wide sweep ranges can generate very many run subfolders. The '_patched' suffix and internal comments ('Added in v25') indicate this is one iteration in a versioned series (v18 through at least v25) of the same core script.

2. Master Configuration File

The hand-edited YAML file that parameterizes a run of the core simulation engine.

`input.yaml`

PURPOSE

The master YAML configuration file driving the solar-system + black-hole flyby N-body simulation (noted in-file as 'Compatible with solar_system_bh_rebound13.py', and used more broadly by later versions such as solar_system_bh_rebound25_patched.py). Centralizes all physical, integration, and output parameters for a run or parameter sweep.

INPUTS

This file is itself the input read by the main simulation script; it is hand-edited and has no upstream dependencies.

OUTPUTS

Not applicable (it is a config file), though a trailing comment block in the file documents the full output directory/file-naming convention produced by the simulation it configures: outputs land under simulations/YYYYMMDD_HHMMSS/<STAMP>__rp<...>__vinf<...>/ containing the input YAML copy, NPZ snapshots, an orbits workbook, belt-run CSVs/PNGs, a hazard summary text file, and a heliocentric plot PNG.

KEY TECHNIQUES

Not applicable (configuration only), but the values it sets imply: Skyfield ephemeris usage (DE440s kernel) for epoch/state initialization; hyperbolic Keplerian orbital elements for the BH (periapsis rp, v-infinity, inclination, time-of-periapsis offset, longitude of ascending node, argument of periapsis) with optional parameter-sweep ranges; REBOUND hybrid integrator settings; asteroid-belt tracer particle generation; a power-law size-frequency hazard/impact model; REBOUND's built-in WebGL visualizer toggle; SimulationArchive replay settings.

NOTES

Notable current values: epoch 1873-09-01T00:00:00Z, kernel de440s, bh_mass_msun 0.1, bh_rp_au given as '0.25, 1.5, 0.25' (a sweep-like triple), bh_vinf_kms 25.0, duration_days 112420 (about 307.7 years), dt_days 0.01, n_belt 0 (belt tracers disabled in this particular config), archive_enable false, write_npz / write_mp4 false, write_uncertainty_csv true with uncertainty_bodies: all -- this flag is presumably what feeds the fiducial.csv / lon_plus.csv / etc. files consumed by the estimate_BH_parameters_uncertainties* scripts. Also sets bh_state_fixed_mu, move_to_com, and debug_initial_state flags controlling coordinate-frame/debug behavior at t=0.

3. Orbit & Physics Post-Processing

Scripts that consume a completed run's output files and derive higher-level physical results: orbital-element changes, black-hole parameter recoverability, impact hazard, Earth temperature effects, and relativistic tidal illustrations.

---

post_orbit_compare_geocentric_fixed4.py

PURPOSE

Compares 'unperturbed' (no black hole) baseline orbits against the actual perturbed orbits from a completed simulation run, for each planet plus the Moon, both heliocentrically and (for the Moon) geocentrically. Quantifies and visualizes how much each body's orbit changed due to the BH flyby by re-integrating a clean baseline simulation and overlaying it against the perturbed trajectory taken from the run's saved Excel workbook.

INPUTS

A run subfolder path (positional CLI arg) containing a __orbits__*.xlsx workbook (per-body sheets with time_days, position, and velocity columns) and an __input.yaml file (for epoch, kernel, dt_days). Also loads a Skyfield/JPL ephemeris kernel (e.g. de440s .bsp) to rebuild the baseline (BH-free) simulation at epoch. CLI args: --tend (default 36500 days), --baseline-step (default 0.5 days).

OUTPUTS

Per-body PNGs (orbit_compare_<Body>.png) comparing one baseline orbit period at epoch vs. the last perturbed orbit near --tend; combined plots orbitcompare__ALL_epoch.png, orbitcompare__ALL_final.png, orbitcompare__Earth_Moon.png, and orbitcompare__Moon_geocentric.png (Moon relative to Earth, baseline vs. final lunar orbit).

KEY TECHNIQUES

Rebuilds a clean N-body REBOUND simulation (WHFast integrator) from Skyfield ephemeris state vectors at epoch to generate the 'unperturbed' reference orbit. Computes osculating orbital elements (a, e, angular momentum) from Cartesian state vectors via vis-viva/eccentricity-vector formulas, infers orbital periods via Kepler's third law, and determines the sampling window for the 'last orbit' using the inferred period. For the Moon, computes geocentric (Moon-Earth relative) osculating elements using the combined Earth+Moon mu.

NOTES

Depends entirely on outputs of the core simulation script (needs the __orbits__*.xlsx and __input.yaml in the run folder) -- not standalone. Contains a duplicated helper-function definition (_add_params_box, defined twice, the second silently shadowing the first). The 'fixed4' name implies prior versions had bugs that were patched.

estimate_BH_parameters_uncertainties.py

PURPOSE

Computes time-evolving uncertainties on black hole parameters -- mass, sky longitude, and sky latitude -- that could be inferred from a solar-system tidal-signal detection, using a Fisher-matrix (linearized least-squares) approach. Answers 'how well could we localize/weigh the BH as a function of time/BH distance, given a set of finite-difference perturbed simulation runs.' Follows the main simulation in the analysis pipeline to assess detectability and parameter-recovery precision of the tidal perturbation signal.

INPUTS

Five (or seven) required CSV files from a fixed 'SolarSystem/uncertainties/' base directory: fiducial.csv, lon_plus.csv, lon_minus.csv, lat_plus.csv, lat_minus.csv (and optionally mass_plus.csv / mass_minus.csv), each with columns time_days, 'Delta_tidal (m)', 'BH_Sun_d (AU)' -- representing the fiducial run plus runs perturbed by small steps around it. CLI args specify file names, perturbation step sizes, fiducial mass, assumed observational noise, sampling cadence, correlation length, minimum points before solving, an optional epoch year for a sky map, and an output-file prefix.

OUTPUTS

A CSV of sigma values vs time (sigmas.csv), a 4-panel PNG (BH distance, sigma-mass in Jupiter masses, sigma-longitude/latitude in degrees, cumulative SNR with 3/5/10-sigma threshold lines), and optionally a sky-map PNG showing a delta-chi-squared contour map (1/2/3 sigma) around the best-fit sky position at a chosen epoch. Prints SNR threshold crossing times/distances and final sigma values to stdout.

KEY TECHNIQUES

Central-difference numerical partial derivatives to build a design matrix of signal-vs-parameter sensitivity; Fisher information matrix accumulated over increasing time windows with robust matrix inversion (falling back to a pseudo-inverse); cumulative matched-filter-style SNR calculation; local quadratic chi-square sky-map construction from a 2x2 covariance submatrix; multi-panel matplotlib plotting.

NOTES

Standalone CLI tool. Depends on upstream CSV outputs presumably generated by the main simulation script run multiple times with perturbed BH sky-position/mass parameters. Uses a fixed relative base directory, unlike the 'fast' variant, which derives its base directory from the script's own location. Effectively an earlier/slower version superseded by estimate_BH_parameters_uncertainties_fast.py (same CLI interface, near-identical outputs).

estimate_BH_parameters_uncertainties_fast.py

PURPOSE

A faster reimplementation of estimate_BH_parameters_uncertainties.py that computes the same Fisher-matrix-based time-evolving uncertainties on BH mass, sky longitude, and sky latitude from perturbed-run CSVs, but uses vectorized cumulative-sum ('prefix sum') math instead of rebuilding matrices from scratch at every time step, for speed on long time series.

INPUTS

Same five/seven required CSV files as the non-fast version, read from a base directory relative to the script's own location (more portable than the base version's hardcoded relative path). Same CLI options as the non-fast script, with the mass parameter always included in the design matrix and slightly different defaults (e.g. longer correlation length, higher minimum point count).

OUTPUTS

A sigmas.csv (time, BH distance, SNR, sigma-mass in solar and Jupiter masses, sigma-longitude/latitude), a 4-panel uncertainties PNG (BH distance, log-scale sigma-mass, log-scale sigma-longitude/latitude, cumulative SNR with threshold lines), and optionally a sky-map PNG. Prints SNR threshold crossings and final sigma values to stdout.

KEY TECHNIQUES

Vectorized Fisher-matrix accumulation via per-timestep outer products and cumulative sums rather than a per-index full recomputation -- this is the 'fast' optimization versus the base script's loop-based matrix rebuilds; central-difference derivatives; robust matrix inversion with pseudo-inverse fallback; local chi-square sky map; matplotlib plotting.

NOTES

Standalone CLI tool, functionally a performance-optimized drop-in sibling of estimate_BH_parameters_uncertainties.py (same purpose, same required CSVs, same output naming convention), intended to be run against outputs of the perturbed-parameter simulation sweeps.

postprocess_belt_sizes_and_hazard.py

PURPOSE

Takes before/after asteroid-belt tracer CSVs from a simulation run, assigns each massless tracer a synthetic physical diameter, computes orbital-element deltas, and estimates an aggregate Earth-impact hazard rate scaled up from the small simulated tracer sample to the real asteroid belt population. The hazard-assessment stage of the belt-perturbation analysis pipeline (a nearly identical version of this logic is also baked directly into the core simulation script, suggesting this file may be an earlier standalone precursor).

INPUTS

--folder (a simulation run folder containing belt_run_before.csv and belt_run_after.csv, with orbital-element columns before/after the flyby). Optional args: assumed impact probability, assumed real number of >=1km main-belt objects, hazard time horizon in years, random seed, and an option to also save the histogram as SVG.

OUTPUTS

belt_run_deltas.csv (per-tracer before/after elements plus a synthetic diameter and deltas), hazard_summary.txt (text report of Earth-crossing fraction, scaled real-belt crosser count, Poisson-model aggregate impact rate, mean waiting time, and probability within the horizon), and a log-binned diameter histogram of Earth-crossers as CSV/PNG(/SVG).

KEY TECHNIQUES

Inverse-CDF sampling of a cumulative power-law size distribution (N(>D) ~ D^-2.5); Poisson-process hazard modeling (constant annual rate leading to an exponential waiting-time distribution); log-log histogram binning.

NOTES

Standalone CLI tool, depends on belt CSVs produced by the core simulation script. Defines Earth-crossing as post-flyby perihelion distance under 1.0 AU.

rebuild_planets_run_deltas_from_npz.py

PURPOSE

A repair/reconstruction utility that regenerates the planets_run_deltas.csv file (before/after orbital elements for all 8 planets plus Moon) directly from the raw NPZ state-vector snapshots, for runs where an older/incompatible version of the main script only wrote a partial file or omitted it. Batch-processes every run subfolder under a given parent simulation directory.

INPUTS

A parent simulation folder (positional CLI arg) containing multiple run subfolders. Within each subfolder, it looks for snapshot_*.npz files, taking the earliest and latest snapshot as 'before'/'after'. Optional --overwrite flag.

OUTPUTS

A planets_run_deltas.csv per subfolder with before/after semi-major axis, eccentricity, perihelion, and aphelion for Mercury through Neptune plus Moon.

KEY TECHNIQUES

Reconstructs osculating two-body orbital elements from Cartesian heliocentric state vectors using specific orbital energy and the eccentricity vector, handling both bound (elliptical) and unbound/hyperbolic cases. Assumes a fixed particle ordering in the NPZ snapshot arrays (Sun, 8 planets, Moon, BH, then belt tracers) and uses the Gaussian gravitational constant for AU/day units.

NOTES

Standalone CLI 'rebuild'/backfill utility that depends on NPZ snapshots already written by the core simulation script. Skips subfolders with fewer than 2 snapshots and, by default, skips subfolders where the output CSV already exists unless --overwrite is given.

summarize_earth_temperatures1.py

PURPOSE

Scans all run subfolders under a simulation parent directory, extracts Earth's orbital perihelion/aphelion distance before and after the BH flyby from each run's planets_run_deltas.csv, converts those to idealized equilibrium temperatures, and compiles everything (alongside the BH encounter parameters for that run) into one master summary CSV -- useful for comparing climate impact across an entire parameter sweep.

INPUTS

A parent folder (positional CLI arg). For each subfolder within it: a planets_run_deltas.csv (must contain an Earth row with before/after semi-major axis, eccentricity, perihelion) and an input.yaml (for BH mass, periapsis, v-infinity, inclination).

OUTPUTS

A single earth_temp_summary.csv in the parent folder, with one row per run subfolder containing BH parameters, Earth's orbital elements before and after, and derived perihelion/aphelion temperatures in Kelvin and Celsius.

KEY TECHNIQUES

Idealized blackbody-style equilibrium temperature scaling (T proportional to 1/sqrt(distance), calibrated to 288 K at 1 AU); pandas-based CSV aggregation and multi-column sorting.

NOTES

Standalone CLI tool that depends entirely on outputs already produced by the core simulation script for a full parameter sweep. Silently skips subfolders lacking the expected deltas CSV, an Earth row, or a readable YAML; raises an error only if no runs at all yield Earth data.

`time_dilation_tides.py`

PURPOSE

A standalone, non-CLI physics-illustration script (unrelated to the N-body integration pipeline) that plots two relativistic/tidal effects near a small black hole as a function of distance from the event horizon: differential tidal acceleration on a human-sized object, and gravitational time dilation. An educational supplement exploring what would happen to a human near the specific 0.1-solar-mass BH used elsewhere in the project.

INPUTS

No external files -- all parameters (BH mass of 0.1 solar masses, human height, standard gravity) are hardcoded constants in the script.

OUTPUTS

No files saved -- displays an interactive matplotlib figure with dual y-axes: tidal acceleration in g's on the left, time dilation factor on the right, both versus distance from the event horizon.

KEY TECHNIQUES

Computes the Schwarzschild radius; estimates a 'spaghettification safe distance' by solving for the radius where differential tidal acceleration across a 2-meter body reaches 1g; computes gravitational time dilation via the Schwarzschild metric approximation; twin-axis matplotlib plot.

NOTES

Fully standalone -- no dependencies on any other script's output and no CLI arguments. Provides physical context for the BH mass used elsewhere in the pipeline but is not wired into the main simulation or its data flow.

4. Archive & Data Utilities

Small utilities for inspecting, converting, replaying, or auditing the raw simulation archive/snapshot files produced by the core engine.

`inspect_archive_hashes.py`

PURPOSE

A small diagnostic utility to inspect a REBOUND SimulationArchive file and check which named particle 'hashes' (Sun, BH, planet names, in both capitalized and lowercase forms) are resolvable in the first snapshot. Used to debug/verify particle naming conventions before running other archive-based tools that rely on hash-based particle lookup.

INPUTS

A single positional command-line argument -- an archive filename or path. If not a direct file path, it searches recursively under a 'simulations' directory for a unique file matching that name, raising an error if zero or multiple matches are found.

OUTPUTS

No files written; prints diagnostic text to stdout: the resolved archive path, the first snapshot's simulation time and particle count, and a pass/fail list for each candidate body name indicating whether it resolves successfully.

KEY TECHNIQUES

REBOUND's SimulationArchive API (version-robust loader checking both capitalizations) and particle-hash lookup wrapped in a try/except.

NOTES

Very short, standalone, no argparse -- a flat top-level script. Purely a diagnostic tool, meant to be run manually against a .bin SimulationArchive output from the main simulation (produced only when archive_enable is true in input.yaml).

`replay_archive.py`

PURPOSE

A minimal, hardcoded interactive-visualization launcher: loads a specific REBOUND SimulationArchive snapshot and starts REBOUND's built-in real-time web-based visualizer server, letting the user watch/replay the simulation forward in a browser. Used for interactive exploration/debugging of a completed flyby run rather than generating static output files.

INPUTS

Hardcoded path variables in the script body pointing to a specific run folder and archive .bin file, plus a hardcoded snapshot index (which snapshot to start from) and port number.

OUTPUTS

No files written -- opens a live web visualizer (e.g. at http://localhost:1235) and prints status/loaded-time info to stdout, then loops until interrupted with Ctrl+C.

KEY TECHNIQUES

Restores simulation state directly from an archive checkpoint and uses REBOUND's WebGL-based real-time visualizer (start_server).

NOTES

Not a CLI tool -- paths/port are edited directly in the source before running for each new run (per its own in-file comments). Depends on an archive produced with archive_enable true by the core simulation script.

`convert_npz_to_csv.py`

PURPOSE

Converts the raw binary NPZ snapshot files from one simulation run into per-frame CSV files enriched with visualization metadata (mass, marker size, RGB color) for each body, making the compact n-body state easier to consume by external plotting/visualization tools or spreadsheets than the NPZ format.

INPUTS

A hardcoded run directory containing snapshot_*.npz files, plus the run's copied config file (matched by glob for '*__input.yaml'), from which the BH mass is read via YAML.

OUTPUTS

One CSV per snapshot written to a csv_frames subfolder, with columns for time, body name, index, mass, plot size, RGB color, and full position/velocity. Planet masses are hardcoded constants; Sun mass fixed at 1.0; BH mass pulled from the run's YAML; unnamed extra particles are labeled as tracers and rendered as small gray zero-mass belt objects.

KEY TECHNIQUES

NumPy NPZ ingestion with pickle allowed; PyYAML config loading; static per-body lookup tables for mass/plot-size/color covering the Sun, all planets, Moon, BH, and a generic tracer category.

NOTES

Standalone script with a main(), no argparse -- the run/output directories are hardcoded constants that must be hand-edited per run before execution. Depends on the NPZ snapshots and input.yaml copy produced by the main

simulation script for that run.

`count_planets_run_rows.py`

PURPOSE

A QA/housekeeping utility that scans every subfolder of a given simulation 'parent' folder (a batch of related runs), locates each subfolder's planets_run_deltas.csv output, and records its row count in a chronologically ordered summary -- useful for auditing how many timesteps/rows each run in a batch produced and spotting incomplete or failed runs.

INPUTS

A parent folder (positional CLI arg) expected to contain multiple run subfolders, each holding a planets_run_deltas.csv file produced elsewhere in the pipeline.

OUTPUTS

A row-count summary CSV written inside the parent folder, with columns for subfolder name, creation/modified time, the CSV filename, row count, and a status flag (ok / no_csv_found / read_error).

KEY TECHNIQUES

Filesystem-stat-based chronological ordering (Windows creation time, falling back to modified time); pandas for reading/counting CSV rows; per-file exception handling so one unreadable/missing CSV doesn't abort the whole scan.

NOTES

Standalone CLI tool. In-file comments explicitly note the Windows-specific meaning of the creation-time stat, consistent with this project's Windows environment. Independent of the other scripts but depends on planets_run_deltas.csv files being produced by the main simulation pipeline.

5. Visualization & Animation

Tools for turning a completed run's snapshots or archive into static plots, interactive viewers, and MP4/GIF animations.

`animate_snapshots3D_1.py`

PURPOSE

Renders a 3D animation of the Sun, planets, Moon, black hole, and asteroid-belt tracer particles moving through the flyby, built from the sequence of NPZ snapshot files a simulation run writes to disk. The primary visual-communication tool for a completed simulation run.

INPUTS

A positional run-directory CLI arg containing snapshot_*.npz files (each with a position/velocity array and optional body names). Simulation time in days is parsed from each filename via a regex.

OUTPUTS

An interactive matplotlib window by default; or an MP4 (via FFMpegWriter) or GIF (via PillowWriter) if requested on the command line.

KEY TECHNIQUES

3D scatter animation via matplotlib's mplot3d and FuncAnimation; heliocentric re-centering each frame by subtracting the Sun's position; fixed body ordering/colors/marker sizes for the Sun, 8 planets, Moon, and BH, with any remaining particles rendered as small gray belt tracers; axis limits auto-scaled from the maximum radial extent across all loaded frames.

NOTES

Standalone CLI tool with mutually exclusive --mp4/--gif output options, plus --stride (use every Nth snapshot) and --fps. Depends on the NPZ snapshot output of the main simulation script. MP4 export requires ffmpeg on PATH.

run_animation.py

PURPOSE

An interactive matplotlib-based 2D playback tool for browsing a sequence of snapshot NPZ files from a simulation run as an animated scatter plot, with live keyboard controls for pause, speed, and frame-timing adjustments. Used for casually reviewing/inspecting orbital motion from a completed run without regenerating video files.

INPUTS

A hardcoded snapshot directory of snapshot_*.npz files (each with x/y/optional z position arrays and a time array). Config constants at the top of the file control the snapshot stride, playback speed, minimum frame interval, and point size.

OUTPUTS

No files -- opens an interactive matplotlib window showing an animated scatter of all bodies' positions across snapshots, with a title showing frame index, sim time, and current speed/clamp settings.

KEY TECHNIQUES

FuncAnimation driving frame-by-frame NPZ loading; dynamic re-computation of the animation frame interval based on the actual simulated time-step between snapshots versus a user-adjustable speed multiplier and a minimum-interval clamp; keyboard event handling for pause, speed, and clamp adjustment.

NOTES

Not a CLI tool -- the snapshot directory and other config are hardcoded constants edited directly in source. Depends on NPZ snapshots from the core simulation script. Distinct from its sibling animate_snapshots3D_1.py, which renders the 3D equivalent.

make_inner_3au_videos_from_archive.py

PURPOSE

Generates two MP4 animations (a 2D x-y plot and a 3D plot) from a REBOUND SimulationArchive, showing the inner solar system (default +/-3 AU around the Sun) evolving over time, including the Sun, planets, optionally the Moon, and the BH. Trajectory segments outside the plotted region are broken so only in-region portions of each trail are drawn -- useful for visualizing planetary perturbations during the BH's close approach without the animation's scale being dominated by the BH's much larger excursions.

INPUTS

A REBOUND SimulationArchive .bin file (positional CLI arg, resolved by filename search under ./simulations/ if only a filename is given). CLI args for hash-based particle selection (or index-based fallback for older archives), plus render options for plot radius, frame stride, FPS, and trail length.

OUTPUTS

Two MP4 video files written next to the input archive: an inner xy-orbits animation and an inner 3D-orbits animation.

KEY TECHNIQUES

REBOUND SimulationArchive random-access snapshot indexing; matplotlib's FuncAnimation combined with FFMpegWriter (H.264 codec) for MP4 export, including a 3D Axes3D animation; heliocentric coordinate computation by subtracting the Sun's position each frame; hash-first / index-fallback particle lookup for compatibility across different archive versions; bounded trail buffers with a maximum length; gap-breaking of trail lines when a body exits the plotted region so the line doesn't jump across gaps; console progress/ETA printing.

NOTES

Standalone CLI tool, requires ffmpeg on PATH. Documents two usage modes in its module docstring: hash-first for newer archives, and an index-fallback for older anonymous archives. Depends on archive_enable true / archive.bin

output from the main simulation.

plot_bh_perihelion.py

PURPOSE

Loads a REBOUND SimulationArchive produced by the main flyby simulation and generates a static 2D heliocentric snapshot (Sun + 8 planets + BH) at the moment of the black hole's perihelion passage relative to the Sun. One of several plotting utilities used after a run to illustrate the encounter geometry.

INPUTS

A REBOUND SimulationArchive .bin file (positional CLI arg, resolved by filename search if needed). Optional args for hash-based or index-based particle lookup and a plot half-width in AU.

OUTPUTS

A PNG plot showing the Sun at the origin, planets as scatter points, and the BH marked with an X, saved next to the archive file. Prints the snapshot index, time, and perihelion distance to stdout.

KEY TECHNIQUES

Iterates over all saved SimulationArchive snapshots to find the one with minimum Sun-BH heliocentric separation; matplotlib 2D scatter plot with equal aspect ratio.

NOTES

Standalone tool, depends on an archive already produced by the core simulation script (requires archive_enable true). Implements the same hash-first/index-fallback particle lookup pattern seen in the core simulation script.

gallery_heliocentric_viewer.py

PURPOSE

A simple keyboard-navigable image gallery that displays pre-rendered heliocentric-plot PNGs (one at a time) from all subfolders under a given simulation-run root, letting the user flip through many runs' final heliocentric plots quickly. The simpler predecessor/companion to gallery_heliocentric_replot_viewer_progress_workbook.py.

INPUTS

Recursively globs a root folder (single positional CLI arg) for files matching the heliocentric-plot PNG naming pattern.

OUTPUTS

No files written; opens an interactive matplotlib window displaying each PNG in turn, with a title showing the image index and source folder/filename.

KEY TECHNIQUES

matplotlib image loading with float-range normalization; keyboard event handling for navigation.

NOTES

Standalone tool; depends on heliocentric-plot PNG files produced as an output artifact by the main simulation script. Controls: left/right arrows to navigate, H to toggle a help banner, Q/Esc to quit. Simpler than the 'replot' viewer since it has no pan/zoom/body-selection -- just flips through static images.

gallery_heliocentric_replot_viewer_progress_workbook.py

PURPOSE

An interactive matplotlib-based image gallery/viewer that lets a user step through multiple simulation runs and see each run's heliocentric orbit plot, but instead of showing a pre-rendered PNG, it re-reads each run's raw Excel orbit workbook and redraws the plot live at a user-specified field of view in AU. This avoids the pixelation that results from zooming into a PNG that was originally auto-scaled to include a very distant BH.

INPUTS

Recursively searches a given root directory for orbit Excel workbooks, each expected to contain a Sun sheet plus per-body sheets (planets, Moon, BH by default) with position and time columns. CLI args for field of view, which bodies to show, pan/zoom step size, line width, legend, and progress-reporting toggles.

OUTPUTS

No files are written -- a live interactive matplotlib window only. Prints same-line console progress messages while loading/plotting each body's data from each workbook.

KEY TECHNIQUES

pandas Excel reading of per-body orbit sheets; heliocentric coordinate conversion by subtracting the Sun's position from each body's position; a custom interactive viewer class implementing pan/zoom/reset/legend-toggle/help-toggle controls via matplotlib key-press events; recursive file discovery with de-duplication.

NOTES

Standalone interactive tool, not depending on any other script's direct output beyond the orbit workbooks produced by the main simulation. Controls: left/right = previous/next run, WASD = pan, +/- = zoom, 0 = reset view, H = toggle help, L = toggle legend, Q/Esc = quit.

6. GAIA Astrometric Detectability Analysis

A separate analysis thread that cross-references the simulated black hole's projected sky track against real Gaia DR3 stars, computing whether a passing 0.1-solar-mass black hole would produce a detectable point-lens astrometric microlensing signature under a synthetic Vera C. Rubin Observatory (LSST) observing cadence.

`GAIA_dr3_v1.py`

PURPOSE

Queries the real Gaia DR3 catalog for stars lying along the passing black hole's on-sky track, then computes for each field star the point-lens astrometric centroid shift over a synthetic Rubin-like observing schedule, applies a magnitude-dependent astrometric noise model, and flags which stars would show a >=3-sigma shift on >=5 distinct nights. The 'real sky' companion to the pipeline's simulation -- it identifies actual observable microlensing candidates from real GAIA astrometry rather than simulated field stars.

INPUTS

A synthetic Rubin observing-schedule CSV (timestamps and RA/Dec of each visit); a BH RA/Dec track Excel workbook located under the matching simulations/<run_date>/<run_prefix>/ folder; a live astroquery query against the gaiadr3.gaia_source table (source ID, position, proper motion, parallax, and photometry), magnitude-limited and row-limited.

OUTPUTS

A CSV of all corridor stars with shift statistics, a CSV of stars passing the >=5-night detection rule, and an Excel-safe variant of the pass CSV with long integer source IDs protected from spreadsheet mangling.

KEY TECHNIQUES

astroquery Gaia archive queries with RA-wrap-aware bounding-box filtering; tangent-plane (gnomonic) projection about the median track position; vectorized point-to-polyline minimum-distance computation to build a corridor filter around the BH track; proper-motion propagation of Gaia positions from the Gaia reference epoch to each observation epoch; the point-lens microlensing centroid-shift formula (Einstein radius scaled by the lens-source impact parameter) with the Einstein radius computed from the BH's actual time-varying distance; a magnitude-dependent astrometric noise model; per-visit-to-per-night detection collapsing.

NOTES

Standalone top-to-bottom script (no main()/argparse); a 'USER SETTINGS' block at the top is hand-edited per run. Requires live network access to the Gaia archive. Its pass-catalog CSVs feed astrometric_shift.py, astrometric_shift_option1.py, detectable_shift_on_date.py, and distance_from_bh_on_first_detect.py.

astrometric_shift.py

PURPOSE

For a single Gaia star chosen by source ID, computes and plots the time series of astrometric microlensing centroid shift caused by the passing BH, contrasting the dense synthetic schedule against a Rubin-Observatory-like realistic cadence (paired same-night visits, minimum night spacing). Produces a 'what would the detection curve actually look like' view for one candidate star under survey-realistic sampling.

INPUTS

The synthetic Rubin schedule CSV, the Gaia pass-catalog CSV produced by GAIA_dr3_v1.py, a BH RA/Dec Excel track auto-discovered by filename pattern, and a required CLI argument for the Gaia source ID.

OUTPUTS

Two interactive matplotlib figures (not saved to disk): the absolute centroid-shift magnitude versus time with a 3-sigma detection-threshold line and the peak point marked, and the RA/Dec shift components versus time.

KEY TECHNIQUES

A custom cadence-enforcement routine that buckets visits into observing nights, keeps at most two same-night visits within a maximum pairing separation, then down-selects nights to a fixed stride with a minimum gap; tangent-plane projection centered on the schedule's median position; linear proper-motion propagation of the star; the same point-lens centroid-shift formula used in GAIA_dr3_v1.py, with a fixed 0.1-solar-mass BH.

NOTES

Standalone script (not wrapped in a main guard), run directly with a required --gaia-id argument. Depends on GAIA_dr3_v1.py's pass-catalog output plus the schedule CSV and BH track workbook from the simulation pipeline.

astrometric_shift_option1.py

PURPOSE

Plots the 2D sky-plane trajectory (RA-shift vs Dec-shift, with time-direction arrows) of the astrometric microlensing centroid shift for a chosen Gaia star under a Rubin-like observing cadence, complementing astrometric_shift.py's magnitude-vs-time view by showing the shape/direction of the centroid excursion in the sky plane.

INPUTS

A required Gaia source-ID CLI argument, plus optional args for the schedule CSV, pass-catalog CSV, BH mass, and all of the cadence-enforcement parameters (max pairing separation, night stride, minimum night gap, etc.) exposed as CLI flags. The BH RA/Dec track is auto-discovered by filename pattern.

OUTPUTS

Two interactive matplotlib figures (not saved to disk): the RA/Dec-shift sky-plane track with chronological direction arrows and the peak point marked, and the shift components versus time for the cadence-enforced schedule.

KEY TECHNIQUES

The same night-pairing/stride-downsampling cadence algorithm, tangent-plane projection, proper-motion propagation, and point-lens centroid-shift formula as astrometric_shift.py; matplotlib annotation-based arrows to show time direction along the track.

NOTES

Standalone CLI script wrapped in a main guard, with a fuller argparse interface than astrometric_shift.py (nearly all cadence parameters are exposed as flags rather than hardcoded). Depends on the same upstream Gaia pass catalog, schedule CSV, and BH track workbook; functionally a sky-plane-view variant of astrometric_shift.py.

centroid_motion.py

PURPOSE

A short code fragment (not a full script) that bins per-visit astrometric centroid-shift data into per-night summary statistics -- counting visits per night and computing mean/max shift and RA/Dec components per night, plus an 'effective' nightly detection threshold assuming noise averages down as one over the square root of the visit count. Reads as a downstream aggregation step in the centroid-shift analysis workflow.

INPUTS

Not self-contained -- it references pre-existing in-scope variables (schedule timestamps, RA/Dec shift arrays, shift magnitude, noise sigma, sigma multiplier) that must already be defined by whatever precedes it. Contains no imports, no CLI, and no file-loading code of its own.

OUTPUTS

An in-memory pandas DataFrame of nightly aggregates (visit count, mean/max shift, mean RA/Dec components, and an effective nightly detection threshold). Nothing is written to disk or printed within the fragment itself.

KEY TECHNIQUES

pandas groupby/aggregate by calendar night; a simple 1-over-square-root-N noise-averaging model for a nightly-averaged detection threshold.

NOTES

Not a standalone runnable script -- no imports, no main guard, no I/O -- it depends entirely on variable names matching those computed in astrometric_shift.py / astrometric_shift_option1.py. Reads like a scratch/notebook cell meant to be pasted into or run after one of those scripts.

detectable_shift_on_date.py

PURPOSE

For every star in a Gaia detection-pass catalog, computes the angular separation from the BH and the resulting point-lens astrometric centroid shift on one specific evaluation date (default 2026-05-04), using the BH's interpolated position on that date. Gives a single-day 'snapshot' of lensing strength across the whole candidate catalog, independent of the multi-night detection logic used in GAIA_dr3_v1.py.

INPUTS

The Excel-safe Gaia pass-catalog CSV (required source ID and position columns), an evaluation date, an optional BH RA/Dec track workbook (auto-discovered as the most recently modified matching file under ./simulations if omitted), the assumed BH mass, and an assumed fixed lens distance used for the Einstein-radius calculation.

OUTPUTS

A CSV, sorted by shift magnitude descending, with per-star evaluation date, BH position, angular separation, assumed BH mass and lens distance, Schwarzschild radius, Einstein radius, and the resulting centroid shift on that date; source IDs re-prefixed for spreadsheet safety.

KEY TECHNIQUES

Haversine great-circle angular-separation formula; Schwarzschild radius calculation; a simplified Einstein-radius formula at a fixed assumed lens distance (differing from GAIA_dr3_v1.py, which interpolates the BH's actual time-varying distance rather than assuming a constant value); the point-lens centroid-shift formula; linear interpolation of BH RA/Dec to the evaluation date.

NOTES

Standalone CLI script wrapped in a main guard. Contains leftover debug code (a hardcoded print of a specific source ID) and an f-string construct requiring Python 3.12+ that will raise a SyntaxError on earlier Python versions. Depends on the Gaia pass catalog from GAIA_dr3_v1.py and a BH track workbook from the simulation pipeline.

distance_from_bh_on_first_detect.py

PURPOSE

Filters the Gaia detection-pass catalog to stars whose recorded first-detection date equals a target date (default 2026-05-04), then computes their angular separation from the BH on that date. Acts as a sanity check on the GAIA_dr3_v1.py detection catalog by verifying how physically close the newly detected stars actually were to the lens on their first detection night.

INPUTS

The Excel-safe Gaia pass-catalog CSV (source ID, position, and first-detection-date columns), an optional BH RA/Dec track workbook (auto-discovered as the most recently modified match under ./simulations if omitted), and a target date.

OUTPUTS

A CSV sorted by ascending angular separation (closest first), with per-star position, magnitude, first-detection date, detection-epoch count, distance to the BH track, peak shift/SNR, BH position on the target date, and the computed angular separation in degrees and arcseconds.

KEY TECHNIQUES

The same Haversine great-circle separation formula as detectable_shift_on_date.py; flexible date parsing with a fallback to an explicit format for Excel-style dates; linear interpolation of BH position to the target UTC-midnight date.

NOTES

Standalone CLI script wrapped in a main guard. Shares helper functions (angular separation, BH-track auto-discovery) with detectable_shift_on_date.py but filters by each star's recorded first-detection date rather than evaluating all stars on an arbitrary date. Depends on the pass catalog produced by GAIA_dr3_v1.py and a BH track workbook from the simulation pipeline.

overlay_bh_on_vr_image.py

PURPOSE

Overlays the BH's projected sky trajectory (RA/Dec track) onto a background sky image framed as a 'Vera Rubin window image' -- visualizing where the hypothetical BH would appear to move across the sky relative to a real observed field/image, including direction-of-motion arrows and start/end markers.

INPUTS

A required BH RA/Dec Excel track (located by extracting the leading timestamp from the filename and searching the matching simulations subfolder) and a background sky image (default a named Vera Rubin field JPG). RA/Dec columns are auto-detected among several possible naming conventions, with automatic hours-to-degrees conversion if needed.

OUTPUTS

A PNG image showing the background field with the BH trajectory plotted in cyan, start/end markers, and periodic direction arrows. Also displays the plot interactively.

KEY TECHNIQUES

A simple linear RA/Dec-to-pixel mapping using a hardcoded rectangular sky-bounds box assumed to correspond to the image's corners; PIL image loading; matplotlib arrow annotations for direction-of-travel indicators.

NOTES

Standalone CLI tool. Depends on a BH RA/Dec Excel workbook that must already exist as output from an earlier pipeline stage. The RA/Dec sky-bounds constants are hardcoded to a specific image/field and would need updating for a different background image.

overlay_plot_v2.py

PURPOSE

Produces a detailed sky-track plot combining a synthetic Vera Rubin (LSST) observing schedule, a computed 'detection corridor' around the BH's RA/Dec path, and a catalog of real GAIA DR3 stars that passed a multi-night detection threshold -- visualizing where and when a passing BH's astrometric signature might be detectable against real background stars during a May-November 2026 observing campaign. Ties the simulation output (BH sky track) together with real GAIA astrometry data for observational-feasibility analysis.

INPUTS

Two hardcoded CSV files: a synthetic Rubin/LSST observing-schedule CSV and the real GAIA detection-pass catalog CSV. Hardcoded settings for the corridor half-width, a magnitude limit, a specific Gaia source ID to highlight, and how many brightest stars to label.

OUTPUTS

A single PNG (dpi 200, transparent background) showing the BH track (with direction arrows), a dashed detection-corridor boundary, scatter points for bright Gaia stars, a highlighted target star, and labeled reference dates, all on a dark 'space' themed background. Also displayed interactively.

KEY TECHNIQUES

Tangent-plane (gnomonic-like) projection around the median RA/Dec of the schedule to compute a normal-offset corridor polygon along the BH track (via per-segment tangent/normal vector averaging for smooth corridor edges); pandas UTC datetime handling with nearest-index lookup for key calendar dates; source ID string cleaning to match Gaia catalog formats; dark-themed matplotlib styling.

NOTES

Not a reusable CLI tool -- all filenames and thresholds are hardcoded at the top of the script with no argparse, so it must be edited in place to use different inputs. Depends on outputs of two upstream pipelines: the synthetic Rubin/LSST schedule generator and the GAIA DR3 query/detection-pass filtering step (GAIA_dr3_v1.py). Contains a few leftover debug print statements from interactive development.